



## PyQuest

# Information- & Aufgabenblatt

## Kapitel 6: Eingaben

Das EVA-Prinzip: Daten einlesen, verarbeiten und schön ausgeben.

### Das EVA-Prinzip & input()

---

Fast jedes Programm folgt dem **EVA-Prinzip** – drei Schritte:

- **Eingabe**: Daten kommen ins Programm (z. B. vom Benutzer).
- **Verarbeitung**: das Programm rechnet oder bearbeitet die Daten.
- **Ausgabe**: das Ergebnis wird angezeigt.

Beispiel BMI-Rechner: **Eingabe** Gewicht & Größe → **Verarbeitung** BMI berechnen → **Ausgabe** BMI anzeigen.

Für die **Eingabe** gibt es die Funktion `input()`. Der Text in den Klammern ist die **Frage**, die angezeigt wird. Das Programm wartet dann, bis der Benutzer etwas eintippt und Enter drückt.

```
name = input("Wie heißt du? ")
print("Hallo " + name + "!")
```

Bei Eingabe Ada:

```
Wie heißt du? Ada
Hallo Ada!
```

#### Aufgabe 1: Welche drei Schritte beschreibt das EVA-Prinzip?

- ☐ a) Erstellen, Verändern, Anzeigen
- ☐ b) Eingabe, Verarbeitung, Ausgabe
- ☐ c) Eingeben, Vergleichen, Ausrechnen

**Aufgabe 2: Wofür steht der Text in den Klammern bei `input("Wie heißt du? ")`?**

- ☐ a) Für die Frage, die angezeigt wird
- ☐ b) Für die Antwort der Nutzerin
- ☐ c) Das ist nur Dekoration und wird ignoriert

**Aufgabe 3: Frage mit `input("Wie heißt du? ")` nach dem Namen, speichere ihn in `name` und gib `Hallo NAME!` aus (bei Eingabe `Ada`: `Hallo Ada!`).**

*Dein Code:*

## Mit Eingaben rechnen

Ganz wichtig: `input()` liefert immer Text (`str`) zurück – auch wenn jemand eine Zahl eintippt!

Willst du mit der Eingabe **rechnen**, musst du sie erst mit `int()` (Ganzzahl) oder `float()` (Kommazahl) umwandeln.

Häufig kombiniert man beides in einer Zeile – `input()` innen, `int()` außen:

*Beispiel*

```
alter = int(input("Wie alt bist du? "))  
print("Nächstes Jahr bist du", alter + 1)
```

**Aufgabe 4: Was passiert bei `input("Zahl: ") + 1` ohne Umwandlung?**

- ☐ a) Es funktioniert einwandfrei
- ☐ b) Es gibt einen `TypeError` – Text und Zahl lassen sich nicht direkt verrechnen
- ☐ c) Python wandelt automatisch um



**Aufgabe 5:** Frage Gib eine Zahl ein: ab, wandle die Eingabe in eine Zahl um, verdopple sie und gib Das Doppelte ist X aus (bei Eingabe 5: Das Doppelte ist 10).

Dein Code:

Jetzt eine echte Anwendung aus dem Fitnessstudio: die **maximale Herzfrequenz** beim Training. Die Faustformel lautet **220 - Alter**. Das ist klassisches EVA: Alter eingeben → 220 - Alter rechnen → Ergebnis ausgeben.

**Aufgabe 6:** Frage Alter: ab, wandle in eine Zahl um, berechne die maximale Herzfrequenz (220 - alter) und gib Maximale Herzfrequenz: X aus (bei Eingabe 40: Maximale Herzfrequenz: 180).

Dein Code:

## Schöner ausgeben mit f-Strings

Bisher hast du Text und Variablen mit + oder mit Kommas verbunden. Es geht aber viel eleganter: mit einem **f-String**.

Du schreibst ein kleines **f** direkt vor die Anführungszeichen und setzt Variablen in **geschweifte Klammern { }**:

```
name = "Anna"
alter = 30
print(f"Ich bin {name} und {alter} Jahre alt.")
```

Ausgabe: Ich bin Anna und 30 Jahre alt.

Der große Vorteil: Du musst **nichts umwandeln**. Innerhalb der { } darf sogar direkt gerechnet werden:

```
preis = 5
print(f"Drei Stück kosten {preis * 3} Euro.")
```



Ausgabe: Drei Stück kosten 15 Euro. – kein str(), kein +, viel lesbarer.

**Aufgabe 7: Was gibt dieses Programm aus?**

```
x = 4  
print(f"Das Quadrat ist {x * x}.")
```

- ☐ a) Das Quadrat ist {x \* x}.
- ☐ b) Das Quadrat ist 16.
- ☐ c) Das Quadrat ist 8.

**Aufgabe 8: Vervollständige den f-String, damit der Name eingesetzt wird. (Setze nur den Buchstaben vor die Anführungszeichen.)**

```
print(____"Hallo {name}!")
```

*Dein Code:*

**Aufgabe 9: Frage Name: und Alter: ab (Alter als Zahl). Gib mit einem f-String aus: NAME ist ALTER Jahre alt. (bei Ada / 16: Ada ist 16 Jahre alt.).**

*Dein Code:*



## Escape-Sequenzen

Eine **Escape-Sequenz** ist eine besondere Zeichenkombination in einem Text, die mit einem Backslash \ beginnt. Damit stellst du Zeichen dar, die man sonst schwer in einen String schreiben kann:

- \n – neue Zeile (New Line)
- \t – Tabulator (ein großer, gleichmäßiger Abstand)
- \" – ein Anführungszeichen mitten im Text

Mit \n kannst du **innerhalb eines einzigen print()** einen Zeilenumbruch machen:

*Beispiel*

```
print("Zeile 1\nZeile 2")
```

**Aufgabe 10: Wie viele Zeilen gibt print("A\nB\nC") aus?**

- ☐ a) 1 Zeile
- ☐ b) 3 Zeilen
- ☐ c) Gar keine

Das Zeichen \t (Tabulator) sorgt für einen sauberen Abstand – praktisch für Tabellen. Und wenn du ein **Anführungszeichen** mitten im Text brauchst, nutzt du \", damit Python es nicht als Ende des Strings versteht:

```
print("Er sagte: \"Hallo\"")
```

Ausgabe: Er sagte: "Hallo"

**Aufgabe 11: Gib mit einem einzigen print() und \n diese zwei Zeilen aus:**

```
Python  
ist toll
```

*Dein Code:*